BEGINNER'S GUIDE · DEEP LEARNING / NLP

Attention Mechanism, Explained From Zero

No background needed. We go one small step at a time, from "how does a computer see a word?" to the four C5 discussion questions. Every idea comes with an everyday example.

How to read this guide

Read it in order, because each step uses the one before it. Don't worry if the math looks scary: every formula is also explained in plain words, and all the numbers are small enough to check on a phone calculator.

1. How computers see words
2. The problem: words need context
3. The old way (RNN) and why it struggled
4. The big idea of attention
5. Query, Key, Value
6. Two tools: dot product & softmax
7. A full calculation by hand
8. Where Q, K, V come from (learning)
9. Why divide by $\sqrt{d_k}$
10. Multi-head attention
11. Positional encoding (word order)
12. Masking (no peeking)
13. The Transformer: the big picture
14. Where you use this every day
15. The 4 discussion questions, answered
16. Glossary & self-check quiz

1 How does a computer "see" a word?

Computers only understand **numbers**, so every word is turned into a list of numbers called a **vector** (or **embedding**). You can think of each number as a score for one feature of the word.

Word	Is it alive?	Is it royal?	Is it an object?	Vector
king (ស្តេច)	0.9	1.0	0.0	`[0.9, 1.0, 0.0]`
cat (ឆ្មា)	0.9	0.0	0.0	`[0.9, 0.0, 0.0]`
table (តុ)	0.0	0.0	1.0	`[0.0, 0.0, 1.0]`

Everyday example: GPS coordinates

Phnom Penh is about (11.5, 104.9) and Siem Reap is about (13.4, 103.9). Places that are close together have similar numbers. Word vectors work the same way: words with similar meanings get similar numbers. Real models use hundreds of numbers per word (BERT uses 768), not 3. The computer learns what they mean by itself, so we can't label each one as neatly as in this table.

One more term: a **token** is a piece of text the model reads. It is often a whole word, but it can also be part of a word, especially in Khmer, which has no spaces between words.

2 The problem: a word's meaning depends on its neighbours

The same word can mean different things:

- "I sat on the river bank." → the **side of a river**
- "I withdrew money from the bank." → a **financial institution** (like ABA or ACLEDA)

The vector for "bank" in Step 1 is the same in both sentences. To get the right meaning, the model has to **look at the other words**: "river" in the first sentence, "money" in the second.

Pronouns have the same problem: "The cat sat on the mat because it was tired." To know that "it" means the cat, the model has to connect the two words.

> **The goal**
> Update each word's vector using information from the **relevant** words around it, so that "bank" near "river" ends up with a different vector from "bank" near "money". Attention is the tool that does this.

3 The old way: RNN/LSTM, reading one word at a time

Before 2017, models called **RNNs** (and an improved version called **LSTMs**) read a sentence **from left to right, one word at a time**. They kept a small "memory" and updated it after each word.

> **Everyday example: the telephone game**
> Ten people stand in a line and whisper a message from one to the next. By the end, the start of the message is distorted. RNNs have the same weakness: in long sentences, **early words fade from memory**.

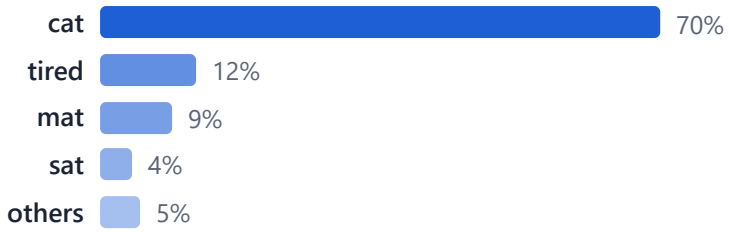
Problem	In simple words
Forgetting (vanishing gradient)	Information from far-back words gets weaker and weaker.
Bottleneck	The whole sentence has to fit into one small memory, like summarizing a book in one sentence.
Slow	Word 10 has to wait for words 1–9, so the work can't be split across a GPU.

4 The big idea of attention: every word can look at every other word

Attention takes a different approach. Instead of passing a message down a line, **every word looks at the whole sentence directly** and decides how much to focus on each other word.

> **Everyday example: a highlighter pen**
> When you read an exam question, you don't give every word equal importance. You highlight the key words. Attention works like that highlighter. For each word, the model gives every other word a **focus percentage**, and the percentages add up to 100%.

Here is how the word **"it"** might divide its attention in "*The cat sat on the mat because it was tired*":



The new meaning of "it" is then built as a **mix**: 70% of cat's information, 12% of tired's, and so on. After this, the vector for "it" "knows" that it refers to a tired cat.

Attention in one sentence

For each word: (1) score how relevant every other word is, (2) turn the scores into percentages, (3) build a new version of the word by mixing information from the others according to those percentages.

Because every word does this **at the same time**, a GPU can compute it in parallel, which makes it fast. And no word is "far away", which fixes the forgetting problem. This is called **self-attention** because the sentence pays attention to itself.

5 Query, Key, Value: the three roles of every word

How does the model decide that "it" should focus on "cat"? It gives every word **three different vectors**, each with its own job:

Query (Q)

"What am I looking for?"

The question a word asks when it wants information.

Key (K)

"What do I have?"

A label that describes the word, so others can decide whether it's relevant.

Value (V)

"Here is my information."

What the word actually passes on if it gets chosen.

Everyday example 1: the classroom

Every word is a student. The student "it" raises a hand and asks: "*Who am I talking about? I need a living noun*". That question is the **Query**. Every student wears a name tag, which is the **Key**: cat's says "living noun", mat's says "object noun", tired's says "feeling". "It" compares its question with every name tag. Cat's tag matches best, so "it" listens mostly to cat and takes its knowledge, which is cat's **Value**.

Everyday example 2: YouTube search

You type "*how to cook Khmer amok*". That's the **Query**. YouTube compares it with every video's title and tags, which are the **Keys**. The best matches rank highest, and then you watch the videos themselves, which are the **Values**. The difference is that attention doesn't pick just one video: it takes a **blend** of all of them, weighted by how well each one matches.

Everyday example 3: hiring for a job

A company posts a job ad saying "need an accountant who speaks English" (**Query**). Candidates send CV headlines (**Keys**). The best-matching candidates are hired and bring their actual skills (**Values**). Notice that the CV headline (Key) and the real skills (Value) are different things. That's why K and V are separate vectors.

Common confusion

Q, K and V are not three different words. **Every single word has all three.** "Cat" has a Q (for when cat asks questions), a K (for when others check whether cat is relevant) and a V (the information cat gives). Every word asks and answers at the same time.

6 Two small math tools: dot product and softmax

Tool 1: the dot product ("how much do two lists agree?")

To compare a Query with a Key, multiply the numbers in matching positions and add them up:

$$[2, 0] \cdot [1, 3] = (2\times1) + (0\times3) = 2$$

Everyday example: matching friends

Each person rates how much they like [football, music, movies] from 0 to 3.
You = [3, 0, 2]. Dara = [3, 1, 2] → 3×3 + 0×1 + 2×2 = **13** (high, so you have a lot in common).
Sokha = [0, 3, 0] → 0 + 0 + 0 = **0** (nothing in common).
Big dot product = good match. That's all "$Q \cdot K^T$" means in the formula: the dot product of every Query with every Key.

Tool 2: softmax ("turn scores into percentages")

Dot products give raw scores such as 4, 0 and 2. Softmax turns them into percentages that add up to 100%:

1. Raise **e** (≈ 2.718) to the power of each score: $e^4 = 54.6$, $e^0 = 1$, $e^2 = 7.4$
2. Add them up: 54.6 + 1 + 7.4 = 63.0
3. Divide each one by the total: 54.6/63 = **87%**, 1/63 = **2%**, 7.4/63 = **12%** (rounded)

Why use e^x instead of just dividing?

(1) It makes every value positive, even when a score is negative. (2) It gives extra weight to the winner, so the best match stands out clearly. That's where the "max" in "soft-max" comes from.

7 A full calculation by hand, step by step

A tiny sentence with 3 words: `cat` `mat` `it`. We'll work out the new meaning of **"it"**. To keep things simple, each vector has only 2 numbers.

Word	Key (K): "what I am"	Value (V): [animal, furniture]
cat	`[2, 0]` (living thing)	`[1, 0]` (fully animal)
mat	`[0, 2]` (object)	`[0, 1]` (fully furniture)
it	`[1, 1]` (unclear)	`[0.5, 0.5]` (unclear)

The Query of "it" is `[2, 0]`, which means "I'm looking for a living thing."

① Score (Q·K)
cat: 2×2+0×0 = **4**

mat: 2×0+0×2 = **0**

it: 2×1+0×1 = **2**

→

② Scale (÷√2 ≈ 1.414)
cat: **2.83**

mat: **0**

it: **1.41**

→

③ Softmax
$e^{2.83}$ = 16.9
e^{0} = 1
$e^{1.41}$ = 4.1
total = 22.0
cat → **77%**
mat → **5%**
it → **19%**

→

④ Mix the Values
0.77×[1,0] + 0.05×[0,1] + 0.19×[0.5,0.5] = **[0.86, 0.14]**

What happened?

Before attention, "it" was `[0.5, 0.5]`: 50% animal and 50% furniture, so its meaning was unclear. After attention, "it" is `[0.86, 0.14]`: **86% animal**. It now "knows" it refers to the cat. **That is attention.** The real Transformer does exactly this, just with hundreds of numbers per word and many words at once.

$$\text{Attention}(Q, K, V) = \text{softmax}(Q \cdot K^{T} / \sqrt{d_k}) \cdot V$$

① score → ② scale → ③ softmax → ④ mix the Values. (d_k = how many numbers are in each Key; here d_k = 2.)

8 Where do Q, K and V come from? (How the model learns)

Each word starts with one vector (from Step 1). The model multiplies it by **three different weight tables**, called matrices, to get its Q, K and V:

$$Q = X \cdot W_Q \quad K = X \cdot W_K \quad V = X \cdot W_V$$

At the start, W_Q, W_K and W_V are **random numbers**. The model learns good values by **training**:

1. The model reads a sentence with a hidden word, for example "The cat sat on the ___", and makes a guess.
2. It checks how wrong the guess was. This is called the **loss**.
3. It works out which direction to nudge every number to be a little less wrong. This direction is called the **gradient**.
4. It adjusts the numbers slightly, then repeats this billions of times.

Everyday example: learning to shoot a basketball

You shoot and miss to the left. Your coach says "a bit more to the right", and that feedback is the gradient. You adjust and try again. After thousands of shots, you're accurate. **If the coach whispers so quietly that you can't hear**, you never improve. Remember this, because it's exactly the "vanishing gradient" problem in the next step.

9 Why divide by $\sqrt{d_k}$? (Discussion Question 1)

In real models each Key has many numbers, for example $d_k = 64$. A dot product adds up 64 multiplications, so scores can become **very large**, like 8 or −8, instead of small numbers like 1 or −1. Look what softmax does with large scores:

Scores for 3 words	Softmax result	What it means
Not scaled: [8, 0, −8]	[**99.97%**, 0.03%, 0.00%]	All-or-nothing. The model is "frozen".
Scaled ÷√64 = ÷8: [1, 0, −1]	[**66.5%**, 24.5%, 9.0%]	A balanced mix. The model can still adjust.

When softmax gives about 100% to one word, a small change in the scores barely changes the output. So the feedback signal (the gradient) is almost **zero**, about 0.0003 instead of about 0.22. That's the coach whispering too quietly: **the model stops learning**. This is the **vanishing gradient** problem.

Why exactly $\sqrt{d_k}$?

If the numbers in Q and K are typical (average 0, spread 1), a dot product of d_k numbers has a spread (variance) of d_k, which means its typical size is $\sqrt{d_k}$. Dividing by $\sqrt{d_k}$ brings the typical size back to about 1. It's a volume knob that is set automatically, whatever the size of the model.

Everyday example: a microphone turned up too loud

If the volume is too high, every sound hits the maximum and "clips", so you can't hear any difference between a whisper and a shout. Dividing by $\sqrt{d_k}$ turns the volume down so the differences can be heard again.

10 Multi-head attention: several highlighters at once

One attention calculation can only focus on one type of relationship. So the Transformer runs **several attention "heads" in parallel** (8 in the original Transformer, 12 in BERT). Each head has its own W_Q, W_K and W_V, so it learns to look for different things. Then their results are joined together.

Head 1: grammar	Head 2: references	Head 3: neighbours
"sat" looks at "cat" (who sat?)	"it" looks at "cat" (what is "it"?)	each word looks at the word next to it

Everyday example: a team of doctors

A heart doctor, a bone doctor and an eye doctor examine the same patient, and each notices different things. The final diagnosis combines all their reports. In the formula: *MultiHead* = *Concat(head$_1$, …, head$_h$)* · W_O, which means "put all the reports side by side, then combine them."

11 Positional encoding: telling the model the word order (Discussion Question 2)

Here is a surprising fact: **attention on its own doesn't know word order.** In the calculation in Step 7, the order of the words never mattered. We only compared vectors. So to attention, a sentence is just a **bag of words**.

Why that's a problem: a Khmer example

«ស្តេចទៅបាញ់សត្វ» = the **king** goes to shoot the **animal**

«សត្វទៅបាញ់ស្តេច» = the **animal** goes to shoot the **king**

Same four words, opposite meaning. Without order information, the model sees **the same thing** for both. This is especially serious for Khmer, because Khmer has no word endings to mark who did what. English has "he" vs "him", and Khmer has no such change. **Word order carries the meaning.**

The fix: before attention, add a special "position vector" to each word's vector. Position 1 gets one pattern, position 2 another, and so on. The patterns are made from sine and cosine waves, which give every position a unique "fingerprint":

$$\mathrm{PE}(\mathrm{pos}, 2\mathrm{i}) = \sin(\mathrm{pos} / 10000^{2\mathrm{i}/\mathrm{d}}) \quad \mathrm{PE}(\mathrm{pos}, 2\mathrm{i}+1) = \cos(\mathrm{pos} / 10000^{2\mathrm{i}/\mathrm{d}})$$

Everyday example: seat numbers at a concert

The same people sitting in different seats make a different arrangement. Positional encoding gives every word a seat number, so "king in seat 1" is different from "king in seat 4". You don't need to memorize the sine/cosine formula. Just remember that it's a unique "address tag" for each position.

12 Masking: no peeking at the future

When a model **writes** text (like ChatGPT), it produces one word at a time. While it's writing word 5, it must not look at word 6, which doesn't exist yet. A **causal mask** blocks attention to future words by setting their scores to −∞ (in practice a very large negative number such as −1,000,000,000), so after softmax they get 0%.

Everyday example

A teacher covers the answer key during an exam. Another example is your phone keyboard suggesting the next word: it can only use what you've already typed.

13 The Transformer: putting it all together

A **Transformer** is simply a stack of the pieces you've just learned, repeated many times (for example 12 layers):

Words → vectors (Step 1) → **+ Positions** (Step 11) → **Multi-head attention** words share information (Steps 4–10) → **Feed-forward** each word "thinks" about what it gathered → **Repeat × N layers** deeper understanding each time

It's like reading a difficult text several times. The first read gets the basic grammar, and later reads get the deeper meaning. Early layers tend to capture **position and neighbouring words**, and deeper layers capture **meaning**.

Encoder: the "reader"

Reads the whole input and **understands** it. Every word can see every other word.

Decoder: the "writer"

Generates output one word at a time, using a mask (Step 12). It can also look at the encoder's output (cross-attention).

Type	Famous models	Good at
Encoder only	**BERT**, **XLM-R** (supports Khmer)	Understanding: classifying, searching, finding names
Decoder only	**GPT** (ChatGPT), **Claude**, LLaMA	Writing and chatting
Encoder + Decoder	Original Transformer (2017), T5	Translating (Khmer → English), summarizing

Remember for the exam

BERT is **encoder-only**. It is not a full encoder–decoder model.

14 Where you already use attention every day

Product	How attention helps
ChatGPT / Claude	Every word it writes pays attention to your whole question and to everything it has written so far.
Google Translate (Khmer ↔ English)	When it writes an English word, it attends to the matching Khmer words, even though the word order differs.
Google Search	Understands that "bank" in "river bank fishing spots" isn't about money.
YouTube / Facebook auto-captions	Speech models use attention to link sounds to words.
Phone keyboard suggestions	Predicts the next word from what you've typed (with a mask).
Spam filters, comment moderation	Focus on the suspicious words in a message.

15 The four discussion questions: beginner answers

For each question there's a plain explanation first, then a purple box with an answer you can say in class.

Q1. ហេតុអ្វីត្រូវប្រើ Scaling Factor $\sqrt{d_k}$? Why $\sqrt{d_k}$, and how does it prevent vanishing gradients?

Plain: Big vectors produce big scores. Big scores make softmax go all-or-nothing (99.97% vs 0%). Then the feedback signal is almost zero and the model can't learn. Dividing by $\sqrt{d_k}$ keeps the scores small and the learning healthy (Step 9).

> **Say it in class**
>
> "The dot product of Q and K has variance equal to d_k, so with large d_k the scores become very large. That pushes softmax into saturation, where its gradient is close to zero, which is the vanishing gradient problem. Dividing by $\sqrt{d_k}$ brings the variance back to 1, keeps softmax in its sensitive range and makes training stable. For example, with $d_k = 64$, the gradient goes from about 0.0003 to about 0.22."

Q2. បើគ្មាន Positional Encoding ...? Without positional encoding, what problems appear?

Plain: Attention can't see word order, so the model reads a "bag of words". «ស្តេចទៅបាញ់សត្វ» and «សត្វទៅបាញ់ស្តេច» look identical to it (Step 11).

> **Say it in class**
>
> "Self-attention is permutation-invariant: it gives the same result whatever the word order. Without positional encoding, the model can't tell the subject from the object, handle negation or question structure, or produce correct word order in translation. This is especially serious for Khmer, which has no inflection and relies on Subject–Verb–Object order to show grammatical roles."

Q3. តើ Attention Weights អាចជាភស្តុតាងនៃវាក្យសម្ព័ន្ធខ្មែរបានទេ? Can attention weights prove Khmer grammar structure?

Plain: Only partly. Tools like **BertViz** draw attention as lines between words. Sometimes a head clearly links a verb to its subject, which is a nice *hint*. But:

- Khmer has no spaces, so the model splits words into pieces (subwords). Attention links *pieces*, not whole words.
- Many heads put most of their attention on special tokens like [CLS] and [SEP], which says nothing about grammar.
- Different heads and layers show different patterns, so you can pick the one that "looks right".
- Attention shows **correlation**, not **cause**.

Say it in class

"Attention weights can be *supporting evidence* but not *proof* of Khmer syntax. Some heads do align with subject–verb links, but subword tokenization, attention to special tokens and variation between heads make the evidence weak. To make a scientific claim, we should compare against a Khmer treebank, use probing classifiers, or do head ablation (switch a head off and measure the effect)."

Q4. តើ Attention Weight តែមួយមុខគ្រប់គ្រាន់ ...? Is attention alone enough to explain the model's reasoning?

Plain: No. Attention shows **where the model looked**, not **why it decided**.

Everyday example: eye-tracking

A camera shows that a student looked at the right line of an exam question. That doesn't prove they understood it, or that this line is why they chose their answer. Attention maps are the model's "eye-tracking".

Say it in class

"No. Attention weights show correlation, not a full explanation. A lot of information also flows through residual connections and feed-forward layers, which attention maps don't show. BERT-base has 144 heads (12 layers × 12 heads), all mixing information together. Research such as 'Attention is not Explanation' (Jain & Wallace, 2019) showed that very different attention patterns can produce the same prediction. Better methods include attention rollout, Integrated Gradients, SHAP/LIME and ablation studies."

16 Glossary: key words in plain English

Term	Simple meaning
Token	A piece of text the model reads: a word or part of a word.
Vector / Embedding	A list of numbers that represents a word's meaning.
Attention	Deciding how much each word should focus on every other word, then mixing their information.
Self-attention	Attention where the words of one sentence look at each other.
Query (Q)	"What am I looking for?"
Key (K)	"What do I contain?", the label that gets matched against queries.
Value (V)	"Here is my information", what gets passed on.
Dot product	Multiply matching numbers and add them up. It measures how well two vectors agree.
Softmax	Turns scores into percentages that add up to 100%.
d_k	How many numbers are in each Key/Query vector.
Gradient	The feedback signal that tells the model how to adjust its numbers.
Vanishing gradient	The feedback becomes almost zero, so the model stops learning.
Saturation	Softmax gives about 100% to one item, so small changes no longer matter.
Head	One attention calculation. Multi-head = several running in parallel.
Layer	One round of attention + feed-forward. Models stack many layers.
Positional encoding	A "seat number" added to each word so the model knows the order.
Mask	Blocks attention to certain words, for example future words.
Encoder / Decoder	The part that reads and understands / the part that writes the output.
BERT / XLM-R	Encoder-only Transformer models. XLM-R supports 100 languages, including Khmer.
BertViz	A tool that draws attention weights as lines between words.
RNN / LSTM	Older models that read one word at a time. Slow, and they forget long-range information.

Self-check quiz

Try answering before you read the answer underneath.

1. In one sentence, what does attention do?

   → For each word, it decides how much to focus on every other word, then builds a new meaning by mixing their information.

2. Match them up: "What do I contain?" / "What am I looking for?" / "Here is my information."
   → Key / Query / Value.
3. What is [3, 1] · [2, 4]?
   → 3×2 + 1×4 = 10.
4. Softmax of the scores [0, 0] gives…?
   → [50%, 50%]. Equal scores give equal attention.
5. What goes wrong if we don't divide by $\sqrt{d_k}$?
   → Scores get too big, softmax becomes all-or-nothing, gradients vanish, and learning stops.
6. Why does the model need positional encoding?
   → Attention alone can't see word order, so "king shoots animal" would equal "animal shoots king".
7. Is BERT an encoder, a decoder, or both?
   → Encoder only.
8. Does a high attention weight prove the model *used* that word to decide?
   → No. It's a clue (correlation), not proof (causation).

The whole lesson in 5 lines

1. Words become number vectors.
2. Each word makes a Query, a Key and a Value.
3. Score = Query · Key, then scale by $\sqrt{d_k}$, then softmax to get percentages.
4. New word meaning = the Values mixed by those percentages.
5. Add positions for word order, use many heads, stack many layers, and you have a Transformer.

Beginner companion to "3-5 Attention Mechanism & Transformer Architecture" (Asia Euro University, Deep Learning / NLP). Main source: Vaswani et al., 2017, "Attention Is All You Need".